Async Credit Processing Service — Design

Components

- **API (FastAPI):** `POST /credits` validates input, registers a pending credit, publishes a Celery task, returns **202** with `task_id`. `GET /credits/{task_id}` reads status from Mongo. `GET /health` pings Mongo.
- **Worker (Celery):** Consumes `process_credit(task_id)` only. Loads amount from the credit document and calls `apply_credit`.
- **MongoDB (replica set `rs0`):** Source of truth for credits and users. Single-node replica set so multi-document transactions work.
- **RabbitMQ:** Celery broker. `acks_late`, prefetch 1, no result backend.

Happy path

1. Client `POST /credits` with `user_id` and `amount` (Decimal, two places).
2. API creates `task_id` (UUID), `register_credit` inserts a pending credit and increments the user's `pending_count` (status `processing`).
3. API `apply_async(task_id)`; returns 202.
4. Worker claims the credit, increments `balance`, decrements `pending_count`, and `settle_status` when idle.

Claim, transaction, and `pending_count`

Idempotency is the claim. In one `with_transaction`:

1. `find_one_and_update({_id, status: "pending"}, set processed)` — first writer wins.
2. On match: `$inc` user `balance` by the credit amount and `pending_count` by `-1` (upsert).
3. `settle_status`: set user `processed` only when `pending_count == 0` **and** `balance` exists.

No match → already settled → return `False`, change nothing. Duplicate delivery credits once.

`fail_credit` uses the same claim pattern to `failed`, decrements `pending_count`, then `settle_status`. If every credit fails, balance never appears, so the user stays **`processing`**.

Money and data model

- `credits`: `_id` = task id, `user_id`, `amount` (Decimal128), `status` (`pending`|`processed`|`failed`), timestamps, `error`.
- `users`: unique `user_id`, `balance` (Decimal128), `pending_count`, `status` (`processing`|`processed`).
- Python uses `Decimal`; Mongo stores `Decimal128`. Never float.

Broker and sync I/O

One sync PyMongo client is shared by API and worker. Endpoints are `sync def`. Celery uses JSON serialization only. Worker retries `PyMongoError` with backoff/jitter (`max_retries=5`); `on_failure` calls `fail_credit`.

Failure windows

- **Publish failure:** if `apply_async` raises `kombu.OperationalError`, API `fail_credit`s, logs, returns **503**. 503 means **unknown** — the broker may have accepted the message before the error. An outbox and client idempotency keys are future work.
- **Registration-to-publish window:** credit is pending in Mongo before publish. A crash after register and before publish leaves a pending credit with no task; operational recovery is out of scope for this service.
- Do not kill a worker after a successful claim mid-transaction; incomplete apply is the dangerous window.

Testing

Real Mongo (DB name must be `credits_test`). No Mongo mocks. Schema validation tests; repository tests including ThreadPoolExecutor concurrent claim; task tests for apply/duplicate/`on_failure`; API tests with `apply_async` monkeypatched (including 503 on `OperationalError`).